PEN :- 240840116021

PRACTICAL-4

Aim:- Perform Exploratory Data Analysis (EDA) using descriptive statistics and identify trends in datasets

Step-1 Import Required Libraries

```
#Import the Pandas library
#Pandas is used for reading, manipulating, and analyzing datasets
import pandas as pd

#Import the Matplotlib library for creating graphs and charts
import matplotlib.pyplot as plt
```

Step-2 Create Dataset

```
data = {
  # Names of the months
  "month":["January", "February", "March", "April", "May", "June", "July", "August", "September", "October", "November", "De
  # Number of units sold in each month
  "units_sold":[120, 135, 150, 145, 160, 175, 180, 170, 190, 210, 230, 250],
  # Total revenue generated in each month (in currency units)
  "revenue":[120000, 135000, 150000, 145000, 160000, 175000, 180000, 170000, 190000, 210000, 230000, 250000],
  # Total profit earned in each month (in currency units)
  "profit":[20000, 23000, 25000, 24000, 27000, 30000, 31000, 29000, 33000, 37000, 41000, 45000]
}
```

Step-3 Create Dataframe

```
# Convert the dictionary 'data' into a Pandas DataFrame
df=pd.DataFrame(data)
# Display the DataFrame
df
```

	month	units_sold	revenue	profit
0	January	120	120000	20000
1	February	135	135000	23000
2	March	150	150000	25000
3	April	145	145000	24000
4	May	160	160000	27000
5	June	175	175000	30000
6	July	180	180000	31000
7	August	170	170000	29000
8	September	190	190000	33000
9	October	210	210000	37000
10	November	230	230000	41000
11	December	250	250000	45000

Step-3 Inspect the Datasaet

```
# Display the first 5 rows of the DataFrame
df.head()
```

	month	units_sold	revenue	profit
0	January	120	120000	20000
1	February	135	135000	23000
2	March	150	150000	25000
3	April	145	145000	24000
4	May	160	160000	27000

```
# Get the number of rows and columns in the DataFrame
df.shape
```

```
(12, 4)
```

```
 #Display the names of all columns in the dataset
df.columns
```

```
Index(['month', 'units_sold', 'revenue', 'profit'], dtype='object')
```

```
# Display information about the DataFrame,
# including columns, data types, and non-null values
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   month       12 non-null     object
 1   units_sold  12 non-null     int64
 2   revenue     12 non-null     int64
 3   profit      12 non-null     int64
dtypes: int64(3), object(1)
memory usage: 516.0+ bytes
```

Step-4 Generate Descriptive Statistic

```
# Generate descriptive statistics for the numerical columns
# Includes count, mean, standard deviation, minimum,
# quartiles (25%, 50%, 75%), and maximum values
df.describe()
```

	units_sold	revenue	profit
count	12.000000	12.000000	12.000000
mean	176.250000	176250.000000	30416.666667
std	38.737168	38737.167963	7525.210155
min	120.000000	120000.000000	20000.000000
25%	148.750000	148750.000000	24750.000000
50%	172.500000	172500.000000	29500.000000
75%	195.000000	195000.000000	34000.000000
max	250.000000	250000.000000	45000.000000

Step-5 Calculate Mean

```
# Calculate the average number of units sold
average_units = df["units_sold"].mean()
# Display the average units sold
print("Average units sold:", average_units)
```

```
Average units sold: 176.25
```

Step-6 Calculate Median

```
# Calculate the median number of units sold
median_units_sold = df["units_sold"].median()
# Display the median units sold
print("Median units sold:", median_units_sold)
```

```
Median units sold: 172.5
```

Step-7 calculate max and min

```
# Find the maximum number of units sold in any month
maximum_units_sold = df["units_sold"].max()
# Display the maximum units sold
print("Maximum units sold:", maximum_units_sold)
```

```
Maximum units sold: 250
```

```
# Find the minimum number of units sold in any month
minimum_units_sold = df["units_sold"].min()
# Display the minimum units sold
print("Minimum units sold:", minimum_units_sold)
```

```
Minimum units sold: 120
```

Step-8 calculate standard deviation

```
# Calculate the standard deviation of units sold
std_units_sold = df["units_sold"].std()
# Display the standard deviation
print("Standard Deviation of units sold:", std_units_sold)
```

```
Standard Deviation of units sold: 38.73716796331634
```

Step-9 Highest

```
# Find the month with the highest revenue
# Display the corresponding month and revenue
highest_sales_index = df["revenue"].idxmax()
highest_sales_month = df.loc[highest_sales_index, "month"]
highest_revenue = df.loc[highest_sales_index, "revenue"]
print("Highest sales month:", highest_sales_month)
print("Highest revenue:", highest_revenue)
```

```
Highest sales month: December
Highest revenue: 250000
```

Step-10 Lowest

```
# Find the month with the lowest revenue
# Display the corresponding month and revenue
lowest_sales_index = df["revenue"].idxmax()
lowest_sales_month = df.loc[lowest_sales_index, "month"]
lowest_revenue = df.loc[lowest_sales_index, "revenue"]
print("Lowest sales month:", lowest_sales_month)
print("Lowest revenue:", lowest_revenue)
```

```
Lowest sales month: December
Lowest revenue: 250000
```

Step 11 calculate total revenue

```
# Calculate the total revenue for all months
total_revenue = df["revenue"].sum()
# Display the total revenue
print("total_revenue:", total_revenue)
```

```
total_revenue: 2115000
```

Step-12 calculate total profit

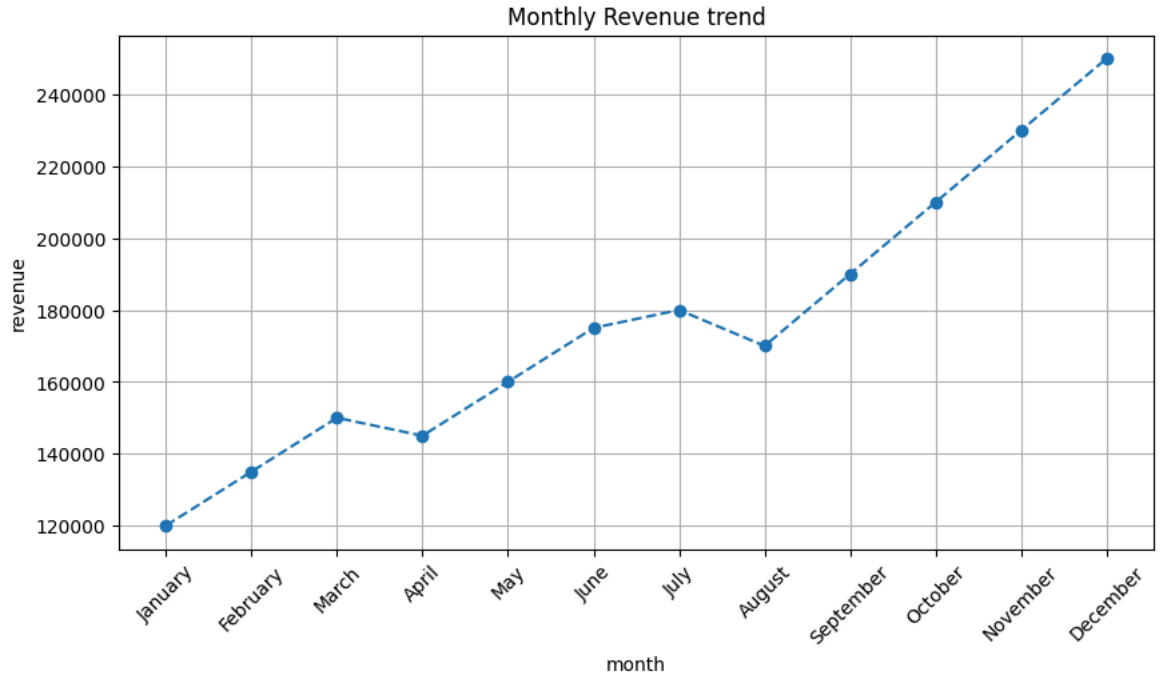
```
# Calculate the total profit for all months
total_profit = df["profit"].sum()
# Display the total profit
print("total_profit:", total_profit)
```

```
total_profit: 365000
```

Step-13 Identify the sales trend

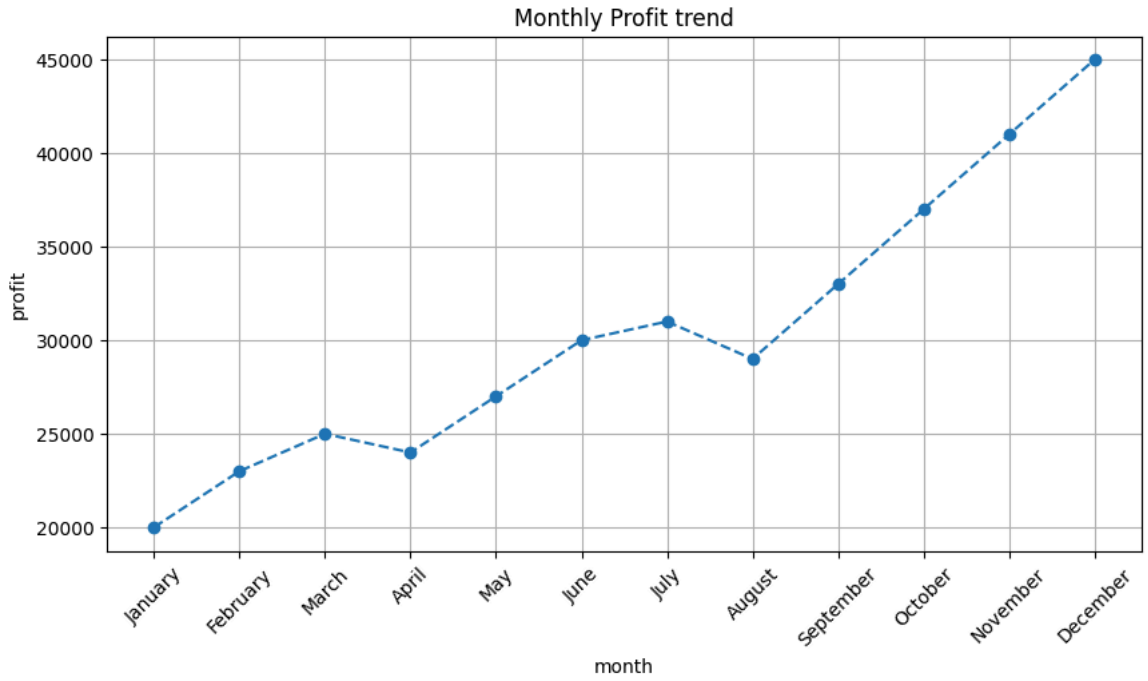
```
# Plot the monthly revenue trend using a line chart
from matplotlib.lines import lineStyles
```

```
plt.figure(figsize=(10, 5))
plt.plot(df["month"], df["revenue"], marker="o", linestyle="--")
# Add chart title and axis labels
plt.title("Monthly Revenue trend")
plt.xlabel("month")
plt.ylabel("revenue")
# Rotate month labels for better readability
plt.xticks(rotation=45)
# Display grid lines
plt.grid(True)
# Show the chart
plt.show()
```



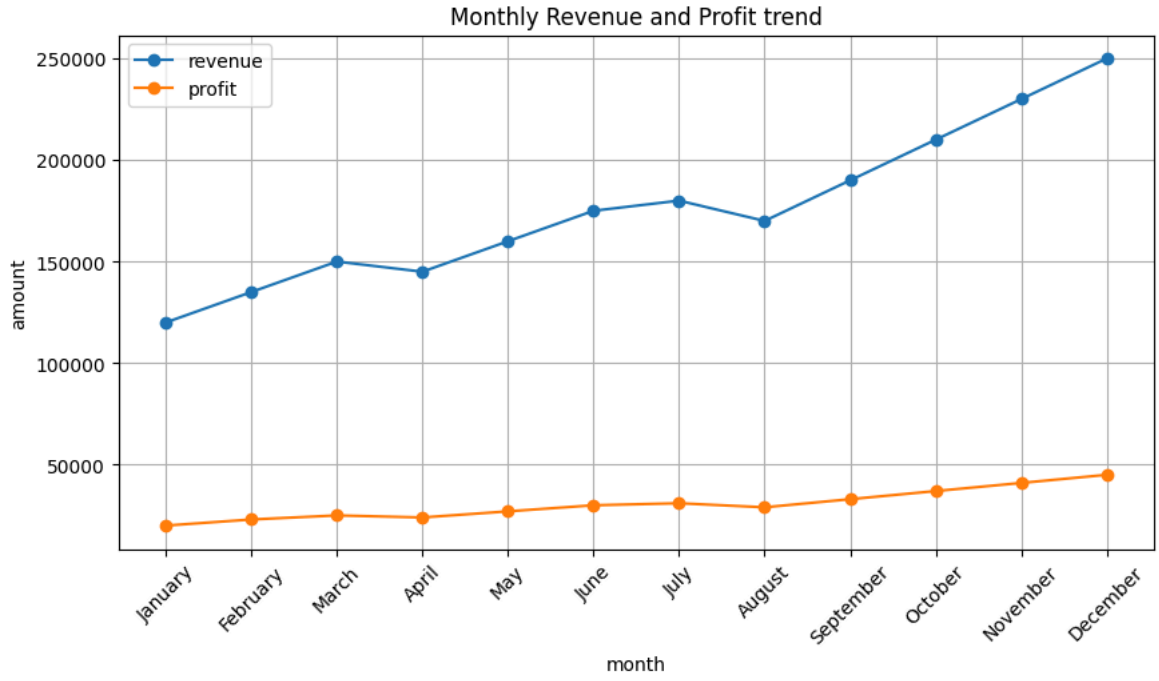
Step-14 Analyze profit trend

```
# Plot the monthly profit trend using a line chart
plt.figure(figsize=(10, 5))
plt.plot(df["month"], df["profit"], marker="o", linestyle="--")
# Add chart title and axis labels
plt.title("Monthly Profit trend")
plt.xlabel("month")
plt.ylabel("profit")
# Rotate month labels for better readability
plt.xticks(rotation=45)
# Display grid lines
plt.grid(True)
# Show the chart
plt.show()
```



Step-15 Compare Revenue and Profit

```
# Plot and compare the monthly revenue
# Profit trends on the same line chart
plt.figure(figsize=(10, 5))
plt.plot(df["month"], df["revenue"], marker="o", label="revenue")
plt.plot(df["month"], df["profit"], marker="o", label="profit")
plt.title("Monthly Revenue and Profit trend")
plt.xlabel("month")
plt.ylabel("amount")
plt.xticks(rotation=45)
plt.grid(True)
plt.legend()
plt.show()
```



Step-16 Identify growth in sales

```
#calculate the % change in revenue
#from one month to the next
df["Revenue_Growth_%"] = df["revenue"].pct_change() *100
df
```

	month	units_sold	revenue	profit	Revenue_Growth_%
0	January	120	120000	20000	NaN
1	February	135	135000	23000	12.500000
2	March	150	150000	25000	11.111111
3	April	145	145000	24000	-3.333333
4	May	160	160000	27000	10.344828
5	June	175	175000	30000	9.375000
6	July	180	180000	31000	2.857143
7	August	170	170000	29000	-5.555556
8	September	190	190000	33000	11.764706
9	October	210	210000	37000	10.526316
10	November	230	230000	41000	9.523810
11	December	250	250000	45000	8.695652

Step-17 identify months with increasing revenue

```
# Filter the rows where revenue growth is greater than 0%
increasing_sales = df[df["Revenue_Growth_%"] > 0]
# Display the filtered DataFrame
df
```

	month	units_sold	revenue	profit	Revenue_Growth_%
0	January	120	120000	20000	NaN
1	February	135	135000	23000	12.500000
2	March	150	150000	25000	11.111111
3	April	145	145000	24000	-3.333333
4	May	160	160000	27000	10.344828
5	June	175	175000	30000	9.375000
6	July	180	180000	31000	2.857143
7	August	170	170000	29000	-5.555556
8	September	190	190000	33000	11.764706
9	October	210	210000	37000	10.526316
10	November	230	230000	41000	9.523810
11	December	250	250000	45000	8.695652

Step-18 save the EDA results

```
# Save the DataFrame as a CSV file without including the index
df.to_csv("EDA_Sales_Analysis.csv", index=False)
# Confirm that the EDA results were saved successfully
print("EDA results save succesfully")
```